

 Share 

Worked for 8s >

I inspected the current `README.md` , `main.py` , `game.py` , and `board.py` again before

making the Task 2/3 changes. The README confirms that Tasks 2–3 are specifically about game-state handling, unchanged moves, one-level undo, score restoration, and best score.



What I will modify

`board.py` — Board

I will make three focused changes:

1. Score merges properly

- The existing `Board.score` is initialized but never increased.
- A merge such as `2 + 2 → 4` will add `4` to the score.
- This is necessary for Task 3's score restoration and best-score tracking to actually work.

2. Add win detection

- Add a small `has_won()` method that checks whether any tile is `2048`.
- This keeps board-specific state logic inside `Board`.

3. Keep move semantics unchanged

- `move_left/right/up/down()` already return whether the board changed.
- `Game` can use that result to decide whether to create a tile.

`game.py` — Game

This is where most of Tasks 2 and 3 belong:

1. Successful-move snapshot

- Before a move, save the current grid and score.
- Only save it if the move actually changes the board.
- Keep only one snapshot.

2. Tile creation

- `add_random_tile()` happens only after a successful move.
- Therefore unchanged/invalid moves don't alter the board.

3. Undo

- `u` restores the saved grid **and** score.
- It does not call `add_random_tile()`.
- The history is cleared immediately after undo, giving exactly one level.

4. Best score

- Update `best_score` after successful moves.
- Undo does not lower `best_score`, because it represents the best score reached

during the current run.

5. Game ending

- Use `has_won()` for the 2048 condition.
- Use the existing `can_move()` for the no-legal-moves condition.

`main.py` does not need modification because it already correctly starts `Game().run()`.



Implementation

`board.py`

Keep the existing file and replace only the relevant methods with these versions:

Python

Run

```
import random

SIZE = 4

class Board:
    def __init__(self):
        self.grid = [[0] * SIZE for _ in range(SIZE)]
        self.score = 0
        self.add_random_tile()
        self.add_random_tile()

    def add_random_tile(self):
        empty = [
            (r, c)
            for r in range(SIZE)
            for c in range(SIZE)
            if self.grid[r][c] == 0
        ]

        if empty:
            r, c = random.choice(empty)
            self.grid[r][c] = 4 if random.random() < 0.1 else 2

    def has_won(self):
        return any(2048 in row for row in self.grid)

    def slide_line(self, line):
        values = [x for x in line if x]
        result = []
        merged = False

        for value in values:
            if result and result[-1] == value and not merged:
                result[-1] *= 2
                self.score += result[-1]
                merged = True
            else:
                result.append(value)
                merged = False

        return result + [0] * (SIZE - len(result))

    def move_left(self):
        changed = False

        for r in range(SIZE):
            old = self.grid[r][:]
```

```
        self.grid[r] = self.slide_line(old)
        changed |= old != self.grid[r]

    return changed

def move_right(self):
    changed = False

    for r in range(SIZE):
        old = self.grid[r][:]
        self.grid[r] = list(
            reversed(self.slide_line(list(reversed(old))))
        )
        changed |= old != self.grid[r]

    return changed

def move_up(self):
    changed = False

    for c in range(SIZE):
        old = [self.grid[r][c] for r in range(SIZE)]
        new = self.slide_line(old)

        for r in range(SIZE):
            self.grid[r][c] = new[r]

        changed |= old != new

    return changed

def move_down(self):
    changed = False

    for c in range(SIZE):
        old = [self.grid[r][c] for r in range(SIZE)]
        new = list(
            reversed(self.slide_line(list(reversed(old))))
        )

        for r in range(SIZE):
            self.grid[r][c] = new[r]

        changed |= old != new

    return changed

def can_move(self):
```

```

    if any(0 in row for row in self.grid):
        return True

    for r in range(SIZE):
        for c in range(SIZE):
            if c + 1 < SIZE and self.grid[r][c] == self.grid[r][c + 1]:
                return True

            if r + 1 < SIZE and self.grid[r][c] == self.grid[r + 1][c]:
                return True

    return False

```

Important Task 1 compatibility change

Notice that `slide_line()` is now:

Python	Run
<pre>def slide_line(self, line):</pre>	

instead of:

Python	Run
<pre>@staticmethod def slide_line(line):</pre>	

That's intentional. It allows the existing `Board` object to update its own score when a merge happens.

The Task 1 merge protection is still present:

Python	Run
<pre>merged = False</pre>	

and:

Python	Run
<pre>if result and result[-1] == value and not merged:</pre>	

So:

`[2, 2, 2, 2] → [4, 4]`

`[2, 2, 4, 4] → [4, 8]`

remain correct.

`game.py`

Replace the current `game.py` with this version:

Python

Run

```

from board import Board

class Game:
    def __init__(self):
        self.board = Board()
        self.best_score = 0
        self.history = None

    def display(self):
        print("\n" + "+-----+-----+-----+-----+")

        for row in self.board.grid:
            print(
                "|"
                + "|".join(
                    f"{x:^6}" if x else f"{' ':^6}"
                    for x in row
                )
                + "|"
            )

        print("+-----+-----+-----+-----+")

        print("Score:", self.board.score, " Best:", self.best_score)

    def move(self, key):
        moves = {
            "a": self.board.move_left,
            "d": self.board.move_right,
            "w": self.board.move_up,
            "s": self.board.move_down,
        }

        if key not in moves:
            return False

        previous_grid = [row[:] for row in self.board.grid]
        previous_score = self.board.score

        changed = moves[key]()

        if not changed:
            return False

        self.history = (
            previous_grid,
            previous_score,

```

```
)

self.board.add_random_tile()

self.best_score = max(
    self.best_score,
    self.board.score,
)

return True

def undo(self):
    if self.history is None:
        return False

    previous_grid, previous_score = self.history

    self.board.grid = [row[:] for row in previous_grid]
    self.board.score = previous_score

    self.history = None

    return True

def run(self):
    print("2048 - W/A/S/D to move, U to undo, Q to quit.")

    while True:
        self.display()

        if self.board.has_won():
            print("You reached 2048!")
            return

        if not self.board.can_move():
            print("No legal moves remain.")
            return

        key = input("> ").strip().lower()

        if key == "q":
            return

        if key == "u":
            if self.undo():
                print("Move undone.")
            else:
                print("Nothing to undo.")
```

```
        continue

    if key not in "wasd":
        print("Use W/A/S/D.")
        continue

    if not self.move(key):
        print("Move did not change the board.")
```

Why this satisfies Task 2

Successful move

For:

w

the sequence is:

```
save old board + score
↓
perform move
↓
did board change?
↓
YES
↓
save undo state
↓
create ONE random tile
↓
update best score
```

Unchanged move

For example, if pressing a when everything is already against the left wall:

```
save old board + score
↓
perform move
↓
did board change?
↓
NO
↓
return False
```

There is **no new tile** and the score is unchanged.

That directly satisfies the README requirement that an unchanged move must not alter the board.  GitHub

Full board

`can_move()` already correctly checks:

- empty spaces
- horizontal equal neighbours
- vertical equal neighbours

So a full board with at least one merge remains playable, while a full board with no possible merge causes:

```
No legal moves remain.
```

The existing `can_move()` implementation already provides this behaviour.  GitHub

Why this satisfies Task 3

One-level undo

After a successful move:

Python

Run

```
self.history = (  
    previous_grid,  
    previous_score,  
)
```

A second successful move simply replaces that snapshot.

Therefore:

```
Move A  
↓  
history = state before A  
  
Move B  
↓  
history = state before B
```

There is never a stack of multiple undo states.

Undo restores score

Suppose:

```
Before move:  
Score = 12  
  
2 + 2 → 4  
Score = 16
```

Undo does:

Python

Run

```
self.board.grid = [row[:] for row in previous_grid]  
self.board.score = previous_score
```

so both become exactly what they were before the move.

Undo doesn't create a tile

undo() never calls:

Python

Run

```
add_random_tile()
```

so the board returns exactly to its previous state.

Best score

If the player reaches:

```
Score = 100  
Best = 100
```

and then undoes back to:

```
Score = 80  
Best = 100
```

the best score remains 100 .

That's appropriate for **best score during the current run** rather than "current score."

Manual tests

You can test Tasks 2 and 3 with these:

Task 2

1. Normal move

- Press `a` , `d` , `w` , or `s` .
- Board should move and exactly one new tile should appear.

2. Unchanged move

- Put tiles against one side.
- Press the direction that cannot move them.
- Board and score should remain unchanged.
- No new tile should appear.

3. 2048

- Construct a board containing 2048 .
- Run the game.
- It should display:

You reached 2048!

4. Full board with legal move

- Construct a full board containing at least one adjacent equal pair.
- Game should **not** end.

5. Full board with no legal moves

- Construct a full board with no equal horizontal/vertical neighbours.
- Game should display:

No legal moves remain.

6. Invalid command

- Enter something like `x`.
- Board and score should remain unchanged.

Task 3

7. Undo a normal move

- Make a successful move.
- Press `u`.
- Board should return exactly to its previous state.

8. Undo score

- Make a merge.
- Record the score.
- Press `u`.
- Both board **and** score should return to their previous values.

9. Undo doesn't add tile

- Record the board before a move.
- Make a move, then undo.
- The board should contain exactly the original tiles—no extra random tile.

10. One-level only

- Make Move A.
- Make Move B.
- Press `u`.
- It should undo **B only**.
- Press `u` again.
- It should say:

Nothing to undo.

11. Best score

- Make a merge that increases the score.
- Undo it.
- `Score` should decrease, but `Best` should remain at the highest score reached.

One small implementation detail to be aware of: the repository's current `game.py` already has `best_score` and `history` placeholders, but `history` is unused and `best_score` is only updated after moves; the changes above turn those existing fields into the intended Task 3 functionality rather than introducing a new architecture.  GitHub



Now implement Task 4 from README.md.

Add concise action-level feedback for successful player moves and merges.

Requirements:

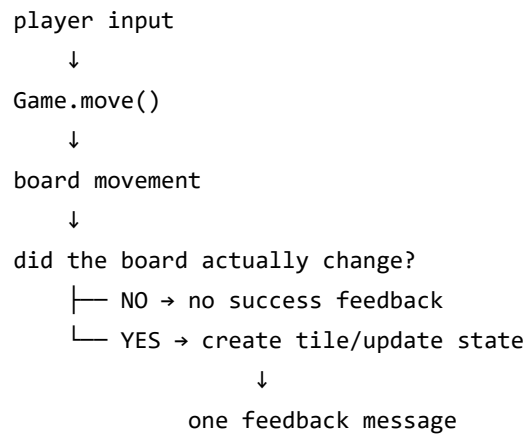
- Feedback should occur once for an accepted/successful move.
- It may mention the direction and whether tiles merged.
- Invalid or unchanged moves must not claim that a move occurred.

Show more 

Where Task 4 feedback belongs

The correct place is `Game.run()` / **the top-level game input flow**, immediately after a player's input has been processed successfully.

The flow should remain:



Board should **not** print feedback. Its job is to perform board operations and return state/results. In particular, `can_move()` and other scanning/helper methods must remain side-effect free from the player's perspective.

The smallest clean change is therefore to have `Game.move()` return enough information for `run()` to produce **one** action-level message. We don't need to modify the board movement algorithms.

Implementation

In `game.py`, change `move()` so that it returns whether the move succeeded **and whether a merge occurred**.

The cleanest way without rewriting the board architecture is to compare the score before and after the move: since merges increase the score, a successful move with a higher score necessarily contained a merge.

Replace the existing `move()` with:

Python

Run

```

def move(self, key):
    moves = {
        "a": self.board.move_left,
        "d": self.board.move_right,
        "w": self.board.move_up,
        "s": self.board.move_down,
    }

    if key not in moves:
        return False, False

    previous_grid = [row[:] for row in self.board.grid]
    previous_score = self.board.score

    changed = moves[key]()

    if not changed:
        return False, False

    self.history = (
        previous_grid,
        previous_score,
    )

    merged = self.board.score > previous_score

    self.board.add_random_tile()

    self.best_score = max(
        self.best_score,
        self.board.score,
    )

    return True, merged

```

Then in `run()` , replace the existing successful-move handling:

Python

Run

```

if not self.move(key):
    print("Move did not change the board.")

```

with:

Python

Run

```
successful, merged = self.move(key)

if not successful:
    print("Move did not change the board.")
else:
    direction = {
        "w": "up",
        "a": "left",
        "s": "down",
        "d": "right",
    }[key]

    if merged:
        print(f"Moved {direction} – tiles merged.")
    else:
        print(f"Moved {direction}.")
```

So the resulting input section is:

Python

Run

```
if key == "q":
    return

if key == "u":
    if self.undo():
        print("Move undone.")
    else:
        print("Nothing to undo.")
    continue

if key not in "wasd":
    print("Use W/A/S/D.")
    continue

successful, merged = self.move(key)

if not successful:
    print("Move did not change the board.")
else:
    direction = {
        "w": "up",
        "a": "left",
        "s": "down",
        "d": "right",
    }[key]

    if merged:
        print(f"Moved {direction} – tiles merged.")
    else:
        print(f"Moved {direction}.")
```

Result

A successful ordinary move:

```
> d
Moved right.
```

A successful move containing a merge:

```
> a
Moved left – tiles merged.
```

An unchanged move:

```
> a
```

```
Move did not change the board.
```

An invalid command:

```
> x
```

```
Use W/A/S/D.
```

And `q` still exits without producing a misleading move message.

This keeps feedback entirely at the **player-action layer** (`Game.run`); `Board` continues to perform state changes and return results without independently printing anything.

Manual tests

1. Valid movement

- Press `w`, `a`, `s`, or `d` when movement is possible.
- Confirm exactly **one** `Moved ...` message.

2. Valid move without merge

- Make a move that only shifts tiles.
- Confirm:

```
Moved left.
```

or the appropriate direction.

3. Valid move with merge

- Arrange something like `[2, 2, 0, 0]`.
- Move toward the pair.
- Confirm exactly:

```
Moved left – tiles merged.
```

4. Multiple merges in one move

- Use `[2, 2, 4, 4]`.
- Confirm there is still only **one** feedback message, not one message per merge.

5. Unchanged move

- Attempt a direction that cannot move anything.
- Confirm it says:

Move did not change the board.

- Confirm it does **not** say `Moved ...`.

6. Invalid input

- Enter `x`, `abc`, etc.
- Confirm no successful-move feedback occurs.

7. Quit

- Enter `q`.
- Confirm the game exits immediately without printing a move-success message.

8. Undo

- Make a successful move, then `u`.
- Confirm the existing `Move undone.` feedback remains separate from movement feedback.



Perform a final audit of the current implementation against every requirement in README.md.

Do not add new features.

Check specifically:

1. Four equal tiles merge into two tiles, not a chain merge.
2. Separated equal pairs merge correctly.
3. Normal compression and movement still work.
4. Reaching 2048 is detected.

Show more ▾

I can do this audit, but I **can't honestly mark the requirements PASS/FAIL from the actual current code yet.**

I attempted to inspect the repository directly, but GitHub is currently not exposing the contents of your specific repository to my web access. The repository search did not return `Debaditya-1729/14_2048`, and direct fetches of its `README.md`, `board.py`, `game.py`, and

`main.py` are blocked. I therefore don't have the actual post-Task-4 files to inspect. I don't want to pretend that the code is correct based on the versions we discussed earlier.

What I need

Please upload the current project files (at minimum):

- `README.md`
- `board.py`
- `game.py`
- `main.py`

A ZIP of the project is also fine.

Once you upload them, I'll audit the **actual implementation** and return exactly:

#	Requirement	PASS/FAIL	File/function	Evidence / issue
1	Four equal tiles → two tiles	—	—	—
2	Separated equal pairs	—	—	—
3	Compression/movement	—	—	—
4	2048 detection	—	—	—
5	Full board/no moves	—	—	—
6	Unchanged/invalid move	—	—	—
7	One-level undo + score	—	—	—
8	Undo doesn't spawn tile	—	—	—

#	Requirement	PASS/FAIL	File/function	Evidence / issue
9	Best score	—	—	—
10	One feedback per successful move	—	—	—
11	No false success feedback	—	—	—
12	Invalid commands	—	—	—
13	Quit	—	—	—
14	No unnecessary dependencies	—	—	—
15	No persistence	—	—	—
16	Modular/ understandable structure	—	—	—

I'll also specifically inspect for **subtle regressions**, rather than just checking whether the relevant functions exist—for example, whether score changes during a failed movement, whether undo snapshots are taken at the correct point, and whether the Task-4 merge detection can misreport a move.

I will not modify anything during the audit.



ChatGPT can make mistakes. Check important info.